

In the Book Recommendation System, each microservice — such as **Book Catalog**, **User Service**, **Recommendation**, and **Rating-Review** — can expose internal metrics like request count, response time, error rates, and resource usage.

Prometheus continuously **scrapes** these metrics (usually from `/metrics` endpoints) and stores them in a time-series database. Grafana then visualizes this data in real-time using dashboards.

This helps in:

- **Tracking performance:** e.g., how fast the Recommendation service responds.
- **Identifying failures:** if Book Catalog or User Service goes down, Prometheus detects it.
- **Monitoring system health:** CPU, memory, and container resource usage.
- **Improving reliability:** by observing traffic patterns and optimizing service connections.

Microservice	What to Monitor (Example Metric)	Why It's Useful
Book Catalog	<code>http_requests_total</code> , <code>response_time_seconds</code>	Detects high traffic or slow catalog responses
User Service	<code>user_logins_total</code> , <code>active_sessions</code>	Tracks user activity and login issues
Recommendation Service	<code>recommendations_served_total</code>	Measures recommendation engine load
Rating-Review	<code>reviews_posted_total</code> , <code>avg_rating_score</code>	Monitors rating/review volume and trends
Frontend (Nginx)	<code>http_requests_total</code> , <code>nginx_up</code>	Tracks request volume and uptime

Current Prometheus and Grafana:

```
shreya-kalgaonkar@shreya-kalgaonkar-Legion-5-16IRX9: ~/Desktop/CCA-2_CCD/monitoring
shreya-kalgaonkar@shreya-kalgaonkar-Legion-5-16IRX9:~/Desktop/CCA-2_CCD$ mkdir monitoring
shreya-kalgaonkar@shreya-kalgaonkar-Legion-5-16IRX9:~/Desktop/CCA-2_CCD$ cd monitoring
shreya-kalgaonkar@shreya-kalgaonkar-Legion-5-16IRX9:~/Desktop/CCA-2_CCD/monitoring$ docker-compose up -d
Creating network "monitoring_default" with the default driver
Pulling prometheus (prom/prometheus):...
latest: Pulling from prom/prometheus
9fa9226be834: Pull complete
1017e2556892: Pull complete
5f4a36c5dba5: Pull complete
964c6bbb921e: Pull complete
62c7956a0b6b: Pull complete
41d61d615b14: Pull complete
f737861e4044: Pull complete
74992c353373: Pull complete
9a21852b2e76: Pull complete
7ee6a1312345: Pull complete
Digest: sha256:23031bfe0e74a13004252caaa74eccd0d62b6c6e7a04711d5b8bf5b7e113adc7
Status: Downloaded newer image for prom/prometheus:latest
Pulling grafana (grafana/grafana):...
latest: Pulling from grafana/grafana
2d35ebdb57d9: Already exists
bd3da438999e: Pull complete
27edc80d437: Pull complete
a5c02213d747: Pull complete
b4cdbc9bc29: Pull complete
0d061a30fff: Pull complete
7d459cad1fcd: Pull complete
f1cfb875b09e: Pull complete
667005ea778d: Pull complete
b4260259b10e: Pull complete
Digest: sha256:35c41e8fd0295f5d0ee5db7e780cf33506abfaf47686196f825364889dee878b
Status: Downloaded newer image for grafana/grafana:latest
Creating prometheus ... done
Creating grafana ... done
shreya-kalgaonkar@shreya-kalgaonkar-Legion-5-16IRX9:~/Desktop/CCA-2_CCD/monitoring$ docker ps
CONTAINER ID   IMAGE             COMMAND                  CREATED        STATUS        PORTS
6be4db397699   grafana/grafana   "/run.sh"                25 seconds ago Up 24 seconds 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
a1ddc4357c97   prom/prometheus   "/bin/prometheus --c..." 25 seconds ago Up 25 seconds 0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp
shreya-kalgaonkar@shreya-kalgaonkar-Legion-5-16IRX9:~/Desktop/CCA-2_CCD/monitoring$
```





